

```

from pathlib import Path
import pandas as pd

# Notebook directory
DATA_DIR = Path.cwd()

```

Environment Setup & Path Management; Function - Imports core libraries (pandas, pathlib), Establishes the notebook's working directory as the data source; Reason - Using Path.cwd() makes the notebook portable and environment-agnostic, Avoids hard-coded file paths, which break across machines, This is the same pattern used in production notebooks and CI pipelines; Business impact - Ensures the analysis can be rerun reliably by reviewers or teammates.

```

ops = pd.read_csv(
    DATA_DIR / "pallet_ops_data.csv",
    parse_dates=["date"]
)

interactions = pd.read_csv(
    DATA_DIR / "pallet_interaction_data.csv",
    parse_dates=["timestamp"]
)

ops.head(), interactions.head()

```

	date	pallet_id	event	declared_value
0	2025-01-06	P0009	Stored	35108
1	2025-01-06	P0005	Checked Out	13180
2	2025-01-06	P0007	Stored	10164
3	2025-01-06	P0011	Stored	24213
4	2025-01-06	P0004	Checked Out	45535,

	timestamp	pallet_id	employee_id	role	action \
0	2025-01-06 11:15:00	P0020	E001	Inbound	Receive
1	2025-01-06 11:15:00	P0020	E001	Inbound	Receive
2	2025-01-06 08:15:00	P0020	E006	Picker	Receive
3	2025-01-06 13:00:00	P0020	E007	Putaway	Audit
4	2025-01-06 15:15:00	P0014	E006	Picker	Audit

	notes
0	FIFO concern
1	NaN
2	NaN
3	Location mismatch
4	Location mismatch)

Data Ingestion; Function - Loads operational pallet data and interaction data, Parses date fields at read time to ensure correct time-based calculations; Reason - Parsing dates upfront avoids downstream casting errors, Keeps raw data ingestion separate from transformation logic; Business impact - Guarantees time-based metrics (storage duration, lifecycle) are accurate.

```

REVENUE_RATE = 0.05
DAILY_STORAGE_COST = 50
ERROR_PENALTY_RATE = 0.10
FIFO_PENALTY = 200

```

Business Parameters; Function - Defines all economic assumptions (rates, costs, penalties); Reason - Centralizes business rules into configurable constants, Enables easy scenario testing without code rewrites; Business impact - Makes the model transparent and adaptable to policy changes.

```

# Separate events
stored = ops.loc[ops["event"] == "Stored"].copy()
checked_out = ops.loc[ops["event"] == "Checked Out"].copy()

# Merge Lifecycle
pallets = stored.merge(
    checked_out,
    on="pallet_id",
    how="left",
    suffixes=("_in", "_out")
)

# Handle pallets never checked out
pallets["end_date"] = pallets["date_out"].fillna(pallets["date_in"])

# Days in storage (minimum 1 day)
pallets["days_in_storage"] = (
    pallets["end_date"] - pallets["date_in"]
).dt.days.clip(lower=1)

pallets.head()

```

	date_in	pallet_id	event_in	declared_value_in	date_out	event_out	declared_value_out	end_date	c
0	2025-01-06	P0009	Stored	35108	2025-01-07	Checked Out	7103.0	2025-01-07	
1	2025-01-06	P0007	Stored	10164	NaT	NaN	NaN	2025-01-06	
2	2025-01-06	P0011	Stored	24213	NaT	NaN	NaN	2025-01-06	
3	2025-01-06	P0019	Stored	34714	NaT	NaN	NaN	2025-01-06	
4	2025-01-07	P0018	Stored	42170	NaT	NaN	NaN	2025-01-07	

Pallet Lifecycle Construction; Function - Separates "Stored" and "Checked Out" events Reconstructs each pallet's lifecycle Calculates storage duration in days; Reason - Operational data is event-based, but economics require state-based modeling Treating missing, checkouts as still-in-storage prevents undercounting costs, Enforcing a minimum of one day

avoids zero-cost anomalies; Business impact - Produces accurate storage cost attribution per pallet.

```
errors = interactions.assign(
    fifo=interactions["notes"].str.contains("FIFO", case=False, na=False),
    mismatch=interactions["notes"].str.contains("mismatch", case=False, na=False)
)

error_flags = (
    errors.groupby("pallet_id")[["fifo", "mismatch"]]
    .any()
    .reset_index()
)

pallets = pallets.merge(
    error_flags,
    on="pallet_id",
    how="left"
).fillna({"fifo": False, "mismatch": False})

pallets[["pallet_id", "fifo", "mismatch"]].head()
```

	pallet_id	fifo	mismatch
0	P0009	True	True
1	P0007	True	True
2	P0011	True	False
3	P0019	True	True
4	P0018	True	True

Error Detection & Flagging; Function - Scans interaction notes for operational error indicators, Aggregates errors to the pallet level; Reason - Errors affect the entire pallet lifecycle, not individual scans, Binary flags prevent compounding penalties from repeated mentions; Business impact - Captures operational risk while keeping penalties fair and interpretable.

```
pallets["revenue"] = pallets["declared_value_in"] * REVENUE_RATE
pallets["storage_cost"] = pallets["days_in_storage"] * DAILY_STORAGE_COST
pallets["error_penalty"] = (
    pallets["declared_value_in"] * ERROR_PENALTY_RATE * pallets["mismatch"]
)
pallets["fifo_penalty"] = FIFO_PENALTY * pallets["fifo"]

pallets["nec"] = (
    pallets["revenue"]
    - pallets["storage_cost"]
    - pallets["error_penalty"]
    - pallets["fifo_penalty"]
)
```

```
pallets[["pallet_id", "nec"]].head()
```

	pallet_id	nec
0	P0009	-2005.40
1	P0007	-758.20
2	P0011	960.65
3	P0019	-1985.70
4	P0018	-2358.50

Economic Metric Calculation (NEC); Function - Calculates revenue, costs, penalties, and Net Economic Contribution (NEC); Reason - NEC provides a single profitability metric per pallet, Allows negative values to highlight loss-generating operations; Business impact - Makes it easy to identify which pallets create or destroy value.

```
# Unique employee touches per pallet
custody = (
    interactions[["pallet_id", "employee_id"]]
    .drop_duplicates()
)

# Touch count per pallet
touch_counts = custody.groupby("pallet_id").size().rename("touches")

# Merge counts
custody = custody.merge(
    touch_counts,
    on="pallet_id"
)

# Equal weight per touch
custody["weight"] = 1 / custody["touches"]

custody.head()
```

	pallet_id	employee_id	touches	weight
0	P0020	E001	6	0.166667
1	P0020	E006	6	0.166667
2	P0020	E007	6	0.166667
3	P0014	E006	4	0.250000
4	P0014	E002	4	0.250000

Worker Custody Weighting; Function - Identifies unique employees who touched each pallet, Assigns equal contribution weights per pallet; Reason - Interaction counts alone can bias

results due to scanning behavior, Equal weighting ensures fairness without role metadata;
Business impact - Enables defensible employee-level performance attribution.

```
worker_impact = custody.merge(  
    pallets[["pallet_id", "nec"]],  
    on="pallet_id",  
    how="left"  
)  
  
worker_impact["impact"] = worker_impact["nec"] * worker_impact["weight"]  
  
worker_impact.head()
```

	pallet_id	employee_id	touches	weight	nec	impact
0	P0020	E001	6	0.166667	NaN	NaN
1	P0020	E006	6	0.166667	NaN	NaN
2	P0020	E007	6	0.166667	NaN	NaN
3	P0014	E006	4	0.250000	NaN	NaN
4	P0014	E002	4	0.250000	NaN	NaN

Worker Impact Attribution; Function - Distributes pallet-level NEC across workers based on custody weights; Reason - Separates value creation from value attribution, Keeps pallet economics independent from employee scoring logic; Business impact - Prevents double counting and preserves financial integrity.

```
final_scores = (  
    worker_impact.groupby("employee_id", as_index=False)["impact"]  
        .sum()  
        .sort_values("impact", ascending=False)  
)  
  
final_scores.head(10)
```

	employee_id	impact
6	E007	844.643333
1	E002	-242.990000
4	E005	-939.417500
2	E003	-1212.667500
0	E001	-1297.800000
5	E006	-1389.117500
3	E004	-1568.250833

Final Worker Scores; Function - Aggregates impact across all pallets per employee, Produces a ranked performance table; Reason - Summation reflects cumulative operational contribution, Sorting enables immediate insight for leadership or dashboards; Business impact - Creates an actionable ranking for recognition or intervention.

```
final_scores.to_csv("worker_scores.csv", index=False)
pallets.to_csv("pallet_nec_details.csv", index=False)
```

Output & Persistence; Function - Exports results for downstream use (Power BI, audits, sharing); Reason - Separates computation from consumption, Ensures results are reproducible outside the notebook; Business impact - Enables BI reporting, stakeholder review, and governance.